

Process Creation

Definition

Process creation is the mechanism by which the operating system starts a new process to perform system work, run a user program, or execute a scheduled job. In practice, a new process is usually created because an **existing process** requests it through a system call.

1. Why Process Creation Is Needed

Main Idea

General-purpose operating systems cannot keep every possible process loaded from the beginning. They must create and terminate processes dynamically as work appears.

Simple vs General-Purpose Systems

- In very simple or single-purpose systems, all required processes may already be present when the system boots.
- In modern general-purpose systems, new processes must be created whenever users, services, or jobs need them.

2. Four Main Events That Create Processes

Process-Creation Triggers

Four principal events cause processes to be created:

1. **System initialization**
2. **A running process makes a process-creation system call**
3. **A user requests a new program**
4. **A batch system starts the next queued job**

3. Creation at System Boot

Boot-Time Processes

When the operating system starts, it usually creates many processes automatically.

Types of Boot-Time Processes

- **Foreground processes:** interact directly with users.
- **Background processes:** perform system services without direct user interaction.

Daemons

Background service processes that wait for events and wake up when needed are called **daemons**. They may handle email, web requests, printing, news delivery, and similar tasks.

Examples

- In UNIX-like systems, running processes can be listed with `ps`.
- In Windows, running processes can be viewed using **Task Manager**.

4. Creation by a Running Process

Purpose

A running process may create one or more additional processes to divide work into smaller, related tasks.

Why This Helps

- It separates responsibilities cleanly.
- It lets multiple activities progress in parallel.
- On multiprocessor systems, different processes may run on different CPUs and improve performance.

Example

One process may fetch a large amount of data from a network and place it into a shared buffer, while another process removes the data from that buffer and processes it. This cleanly separates input work from computation.

5. Creation by User Request

Interactive Systems

Users often create a new process by typing a command in a shell or by clicking an application icon.

UNIX vs Windows Behavior

- In command-based UNIX systems, the new process often takes over the window or terminal in which it was started.
- In Windows, a newly created process does not necessarily begin with a window, but it may create one or more windows afterward.

Practical Result

Users can keep multiple windows open at the same time, each associated with a different process, and switch between them as needed.

6. Creation in Batch Systems

Batch Jobs

In large batch systems, users submit jobs to a queue. When the operating system decides that enough resources are available, it creates a new process and runs the next job.

Where This Appears

This model is especially common in large mainframe and scheduled-processing environments.

7. How Process Creation Happens Internally

Key Mechanism

In every case, a new process is created because an **existing process** executes a **process-creation system call**.

Who May Make the Call?

The caller may be:

- a running user process,
- a system process started by keyboard or mouse activity,
- or a batch manager process.

What the System Call Tells the OS

The call instructs the operating system:

- to create a new process, and
- to determine which program will run inside that new process.

8. UNIX Process Creation

UNIX Model

In UNIX and most POSIX-style systems, process creation usually happens in two steps:

- `fork()` creates the child process.
- `execve()` loads a new program into that child.

8.1 What `fork()` Does

Initial State After `fork()`

`fork()` creates a child that is initially an almost exact clone of the parent. Right after the call:

- parent and child both exist,
- both start with the same memory image,
- both inherit the same environment strings,
- both inherit the same open files.

Why Use Two Steps?

The two-step model lets the child adjust its file descriptors after `fork()` but before `execve()`. This is essential for input, output, and error redirection.

Shell Example

When a user types a command such as `sort`, the shell typically:

- calls `fork()` to create a child,
- then the child calls `execve()` to run `sort`.

9. Windows Process Creation

Windows Model

Windows uses a single Win32 function, `CreateProcess()`, to both create the process and load the target program.

What `CreateProcess()` Can Specify

The call can include:

- the program to execute,
- command-line parameters,
- security attributes,
- whether open files are inherited,
- priority information,
- window settings,
- and a structure used to return information about the new process.

Key Difference

Windows combines process creation and program loading into one API call, while UNIX usually separates them into `fork()` and `execve()`.

10. Address Space and Resource Sharing

Important Rule

After a process is created, parent and child have **distinct address spaces**. A change made in one process's memory is not directly visible in the other.

UNIX Memory Behavior

In UNIX, the child's initial address space begins as a copy of the parent's. However, the two are still separate processes with separate writable memory.

Copy-on-Write

Many UNIX systems use **copy-on-write**. This means memory is not copied immediately. Instead, a memory region is copied only when one of the processes tries to modify it.

What May Still Be Shared

Although writable memory is not shared by default, some resources may still be shared, such as:

- open files,
- read-only program text,
- and selected operating-system-managed resources.

Windows Memory Behavior

In Windows, parent and child begin with different address spaces from the start.

11. UNIX vs Windows Summary

System	Creation Method	Main Idea
UNIX POSIX	/ <code>fork() + execve()</code>	First create a child process, then usually replace the child's memory image with a new program.
Windows	<code>CreateProcess()</code>	One call both creates the process and loads the requested program.

12. Quick Review

Exam-Focused Points

- Process creation allows the OS to start new execution units as work appears.
- The four main triggers are boot, system call, user request, and batch-job initiation.
- **Daemons** are background processes that wait for events and provide services.
- UNIX usually uses `fork()` followed by `execve()`.
- Windows usually uses `CreateProcess()`.
- Parent and child are separate processes with separate writable address spaces.
- **Copy-on-write** improves efficiency by delaying memory copying until modification is necessary.